



IT Genius Engineering
IT GENIUS ENGINEERING CO., LTD.

COURSE ID

GENAI-5D

AI / GENERATIVE AI

COURSE OUTLINE · TRAINING PROGRAM DETAILS

Generative AI for Software Developer

• 30 Hours

• 5 Days

• Level Intermediate-Advanced



Duration: **5 Days**

30 hours (6 hrs/day)



Format: **In-house**

Quote available on request



Course details

www.itgenius.co.th

The year 2026 marks a turning point for software development, shifting from the era of "AI code autocomplete" to the era of **Agentic Software Engineering**, where AI can plan, write, modify, test, and deliver features semi-autonomously under a developer's supervision. Modern developers no longer compete on "how fast they can type code," but on their ability to communicate intent, engineer context (Context Engineering), and direct AI agents to work correctly, safely, and with high quality.

This 5-day edition extends the 3-day edition by adding **two in-depth days on AI Engineering and bringing AI systems to Production (LLMOps)**. It is ideal for those who want to move from "using AI to assist development" to "building real AI-powered applications." It covers LLM fundamentals, Prompt/Context Engineering, AI Coding Agents, connecting via MCP, building AI-core applications (RAG, Tool Calling, AI Agents), through to evaluation, Guardrails/Security, and deploying to production — concluded with a **Capstone Project** in which learners build a real system end to end.

TECH STACK



Claude · Claude Code

ChatGPT



Gemini



Copilot Agent



Cursor

MCP



Python



LangChain

Vector DB

COURSE FEE · IN-HOUSE

Request a Quote

Flat-rate pricing for in-house training · Dates and number of participants by agreement · Quote available on request



ADD ON
LINE

☎ 02-570-8449

☎ 088-807-9770

💬 @itgenius

🌐 www.itgenius.co.th



01 Course Objectives

- Understand the fundamentals of Generative AI, LLMs, Transformers, and Reasoning Models, plus the capabilities and limitations of 2026-era models
- Compare and select leading models appropriately for each task (Claude, GPT, Gemini, and open-source models)
- Advance from Prompt Engineering to Context Engineering to instruct AI to design, write, analyze, and modify code accurately
- Use AI Coding Agents in a real workflow — from planning, writing code, refactoring, and tests, to code review
- Connect AI to organizational tools and data via MCP, and build AI-powered apps with RAG, Tool/Function Calling, and AI Agents
- Evaluate, test, and bring AI systems to production with LLMOps, Evaluation, Observability, and Guardrails

02 Target Audience

- Software Developers across the stack — Full Stack, Backend, and Frontend
- DevOps Engineers and Software Architects who want to bring AI into their workflow
- Those moving into the AI Engineer / AI Application Developer track
- AI Enthusiasts, Technical Leads, and programmers transitioning into the AI-first era
- App developers or freelancers who want to cut development time and cost, plus instructors, trainers, and mentors

03 Prerequisites

- Prior programming experience in any language (Python, JavaScript, TypeScript, Java, etc.) — for Days 4–5, familiarity with Python or Node.js is recommended
- Understanding of basic software principles such as MVC, API, Database, and Git
- Experience with VS Code or another IDE, and with Windows, macOS, or Linux
- Basic English at a read-and-write level (fluency not required)

04 Duration

- **30 hours** · over 5 days (6 hours per day) · hands-on every day, concluding with a Capstone Project

05 Training Topics

- **DAY 1** · Foundations: Generative AI, LLM & Context Engineering (Section 1–3)
- **DAY 2** · Agentic Coding, MCP & Software Design (Section 4–7)
- **DAY 3** · AI Prototyping, Automation, Testing & Best Practices (Section 8–11)
- **DAY 4** · AI Engineering: Building AI-Powered Applications (Section 12–15)
- **DAY 5** · Production, LLMOps, Evaluation & Capstone (Section 16–19)



DAY
1

Foundations: Generative AI, LLM & Context Engineering

Training Content · Section 1-3

TOTAL
6 Hours

1 Generative AI & LLM Landscape in 2026

- The concept of Generative AI and its role in the Software Development Life Cycle (SDLC)
- 2026 model overview: Claude (Opus 4.8 / Sonnet 4.6 / Haiku 4.5), GPT-5.5, Gemini 3.5, and open-source models
- Differences between Rule-based, LLM-based, and Reasoning/Agentic Models
- Choosing the right model: capability, cost (cost/token), speed, context window, and data privacy

2 Understanding LLMs and Transformers in Depth

- The architecture behind LLMs: Transformer, Tokenization, and Context Window
- What Reasoning Models and "Thinking" are, and how they differ from previous generations
- RLHF and Post-training: how they make AI follow instructions and become safer
- Understanding Hallucination and the technical reasons AI gives wrong answers

3 From Prompt Engineering to Context Engineering for Developers

- Principles of good prompts (Clear, Specific, Role-based, Few-shot) and prompt structures for dev tasks (Coding, Debugging, Explain, Refactor, Unit Test)
- Context Engineering: managing context, knowledge files, and system prompts, and writing standard files such as AGENTS.md and CLAUDE.md
- Hands-on Lab — Day 1: Set up ChatGPT, Claude, Gemini + API keys; try Generate/Refactor/Explain and compare models; write your first AGENTS.md file



DAY
2

Agentic Coding, MCP & Software Design

Training Content · Section 4–7

TOTAL
6 Hours

4 AI Coding Assistant & Agentic Coding

- Three forms of AI coding tools: Inline Suggestion → Chat Assistant → Autonomous Agent
- 2026 Agentic Coding tools: Claude Code, Cursor, GitHub Copilot Agent Mode, Windsurf, OpenAI Codex
- Using an agent to plan and build features end-to-end (Plan → Edit → Run → Verify), with techniques to direct agents and review diffs

5 MCP — Connecting AI to Real Tools and Data

- What the Model Context Protocol (MCP) is, and why it's called “the USB-C of the AI world”
- MCP architecture: Host, Client, Server, and communication via JSON-RPC
- Connecting AI to GitHub, databases, file systems, and internal tools, with security and access-control practices

6 AI for Software Design & System Architecture

- Generate UML, ERD, Sequence, and Architecture diagrams from prompts (Mermaid / PlantUML)
- Use AI to design APIs (RESTful / GraphQL), draft OpenAPI specs, and evaluate architectural options (Trade-off Analysis)

7 Legacy Code, Refactoring & AI Code Review

- Have AI explain old code (Explain Code), reverse-engineer, refactor, and add comments/documentation
- AI Code Review: detect bugs, security vulnerabilities, and quality issues before merge
- Hands-on Lab — Day 2: Direct an agent to build API + Database Schema + Diagram, install an MCP Server (GitHub/Filesystem), and have an agent read, refactor, and review legacy code



DAY
3

AI Prototyping, Automation, Testing & Best Practices

Training Content · Section 8–11

TOTAL
6 Hours

8

AI Prototyping & UI Generation

- Use AI to build UIs with HTML, Tailwind CSS, and ready-made components
- 2026 web/UI generation tools from prompts: v0, Lovable, Bolt, Framer AI
- Generate static/full-stack web apps and deploy on GitHub Pages / Vercel / Netlify, then refine the generated code for real use

9

AI-powered Chatbot, RAG & Assistant (Introduction)

- The concept of RAG (Retrieval-Augmented Generation) and why it's needed for enterprise chatbots
- Build a chatbot that answers from documents/knowledge bases, plus FAQ Bot and Coding Assistant with Claude / ChatGPT
- Overview of extending with a Vector Database and Embeddings (groundwork for Day 4)

10

AI Testing, Automation & Spec-driven Development

- Use AI to write Unit Tests, Integration Tests, and generate test data
- Spec-driven Development: write clear specs, then have AI build and verify against the spec, and connect to CI/CD

11

Best Practices, Security & Governance (for Developers)

- Integrate AI into the Dev Lifecycle (Plan → Build → Test → Review → Deploy) with cautions on Security, Hallucination, Privacy, and Copyright
- Hands-on Lab — Day 3: Build a web app/UI from a prompt and deploy to real hosting, build a basic RAG chatbot, and assemble a personal Prompt Library



DAY
4

AI Engineering: Building AI-Powered Applications

Training Content · Section 12–15

TOTAL
6 Hours

12 LLM API & Application Integration

- Fundamentals of calling LLMs via API: Anthropic, OpenAI, and Gemini
- Key parameters: Temperature, Max Tokens, System Prompt, Structured Output (JSON Mode)
- Prompt templates, context-window management, Streaming, Error Handling, Rate Limits, and token cost

13 Production-grade RAG

- Full RAG architecture: Ingest → Chunk → Embed → Store → Retrieve → Generate
- Embeddings and Vector Databases (pgvector, Qdrant, Pinecone, Chroma)
- Improving retrieval quality: chunking strategy, hybrid search, re-ranking; reducing hallucination via citation/grounding

14 Tool/Function Calling & Building AI Agents

- Tool / Function Calling: enabling LLMs to call functions, APIs, and databases
- The AI Agent concept (planning, tool use, looping) and patterns: single-agent, multi-agent, workflow orchestration
- Overview of agent frameworks: LangChain, LlamaIndex, LangGraph

15 Building Your Own MCP Server

- Step up from “user” to “builder”: develop an MCP Server exposing your tools/data to AI agents, designing Tools, Resources, Prompts, and security
- Hands-on Lab — Day 4: Call an LLM API with Structured Output, build a small RAG over learners' own documents, build an AI agent that calls tools, and develop a simple MCP Server



DAY
5

Production, LLMOps, Evaluation & Capstone

Training Content · Section 16–19

TOTAL
6 Hours

16 Evaluation & Testing of AI Systems

- Why AI systems are harder to test than ordinary programs (non-deterministic)
- Designing eval sets and metrics for LLM/RAG, LLM-as-a-Judge, and regression testing when changing models or prompts

17 LLMOps & Observability

- The LLMOps lifecycle: Prompt/Model versioning, deployment, and monitoring
- Tracking cost, latency, and answer quality in production, with logging, tracing, and observability
- Techniques to reduce cost and improve performance: caching, routing, model selection

18 AI Security, Guardrails & Responsible AI

- LLM-specific threats: Prompt Injection, Jailbreak, Data Leakage (OWASP Top 10 for LLM)
- Implementing Guardrails: input/output validation, content filtering, policy, and handling PII
- Responsible AI principles: bias, transparency, and accountability

19 Deployment & Capstone Project

- Deployment approaches (Serverless, Container, Managed Platform) and designing for scalability and reliability
- Capstone Project: build an AI-powered application end-to-end (design → build RAG/Agent → evaluate → add guardrails → deploy) and present the result

INTERESTED IN IN-HOUSE TRAINING · REQUEST A QUOTE

Request a Quote

 02-570-8449

 @itgenius

 088-807-9770

 contact@itgenius.co.th